LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS5068NI– Cloud Computing and the Internet of Things

Project Name

Smart Forest Fire Detection and Alert System

Assessment Weightage & Type

50% Group Coursework

Year and Semester

2026 Spring

Group Members

London Met ID	Student Name
24046536	Ujwal Sharma
24046510	Smith Bhattarai
24046366	Bibek Dangi
24046540	Abhinav Shrestha
23056140	Abinash Kumar Sah
23056224	Shital Kapari

Assignment Due Date: 18 May 2026

Assignment Submission Date: 18 May 2026

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Acknowledgement

I would like to express my gratefulness to everyone who helped me in the process of creating this project.

I would like to thank my module leader Mr Jagannath Paudyal for his invaluable guidance, encouragement and constant support in this project. He guided me in grasping the fundamental concepts of cloud computing and IoT. I would also like to thank my workshop tutor Mr. Subash Sharma for his practical help and useful inputs while implementing this project which helped me achieve the successful completion of this project.

I would like to express gratitude to the college for offering me the requisite resource and atmosphere to fulfil this work. It is with great pleasure that I pronounce my gratitude to my friends for their support and useful discussions, which have helped me improve my ideas and approach.

Lastly, we wish to thank our parents who have helped us so much during this project by constantly being our motivator, being our co-contributor and encouraging us financially. This project has been a great learning experience in that we were able to link theory with reality.

Abstract

The development of Smart Forest Fire Detection and Alert System using the Internet of Things (IoT) and cloud computing is proposed in this project. The objective of this project is to identify fire hazard and alert in real time to reduce environmental and human damage.

It uses sensors such as flame sensor, MQ-2 smoke sensor to sense the environment. The data gathered from the sensors is then sent to a cloud server through Wi-Fi using an ESP32 microcontroller. There is also a GPS module in the system that can give information about the location during the fire detection event. It detects fire, activates local alarms using buzzer, led and connects to a cloud server.

This data is fed to the cloud platform (AWS EC2 with Node-RED) and sent alerts to the user via AWS Simple Notification Service (SNS), including e-mail alerts with fire location information. The system offers an efficient and affordable approach to early detection of fire, facilitating timely action and enhancing safety by leveraging real-time monitoring and automatic alerts.

Table of Contents

Table of Figures

List of Tables

1. Introduction

Forest fires are an environmental problem that is severe and leads to destruction of wildlife, vegetation and human beings. Conventional methods of detection are usually slow and inefficient, which results into delayed response (Alkhatib, 2014). The proposed project is an IoT-based Smart Forest Fire Detection System which is based on sensors and an ESP32 microcontroller to measure and track the fire conditions in real-time. The system transmits data to cloud and generates alerts with location information so that action can be taken promptly (Al-Fuqaha, et al., 2015).

Figure 1: Forest Fire

1.1. Project Overview

The proposed Smart Forest Fire Detection and Alert System is an IoT-based solution designed to detect forest fire incidents at an early stage using smoke and flame sensors connected to an ESP32 microcontroller. The system transmits sensor data through Wi-Fi using the MQTT communication protocol to a cloud-based infrastructure hosted on AWS EC2 with Mosquitto MQTT broker and Node-RED for real-time monitoring and processing. When fire or smoke is detected, the system activates local alerts using LED

and buzzer and sends email notifications with location information through AWS Lambda and SNS services to support rapid emergency response (Jayavardhana, et al., 2013).

1.2. Current Scenario

The problem of forest fires has become a major environmental problem in the world, leading to the destruction of natural resources, wildlife and human settlements. The conventional approaches of fire detection involve manual surveillance or satellite-based surveillance, which often result in delays in detecting the presence of fire. These delays enhance the danger of major extensive destruction and make authorities hard to respond promptly. Automated systems that are capable of offering real time monitoring and early detection of fire incidents are on the increase (Pandey, et al., 2022).

1.3. Problem Statement

The biggest issue with the existing forest fire management systems is the absence of real-time detection and immediate alert systems. Current systems are usually slow, ineffective, and incapable of giving accurate location data. This causes slow reaction of the firefighting teams which causes serious environmental and economic damages. Hence, a real time, automated, and efficient fire detection system is needed.

1.4. Aim and Objectives

The aim of this coursework is to design an IoT solution capable of tracking forest fires in real-time and sending real-time alerts with location data on cloud technology.

Objectives:

- To detect fire using smoke and flame sensors.
- To process sensor data using ESP32 microcontroller.
- To transmit data to a cloud platform via Wi-Fi.
- To generate alerts during fire detection.
- To provide GPS-based location information.
- To enable faster response by notifying authorities

2. Background and System Design

2.1. System Overview

The proposed system is an IoT based Smart Forest Fire Detection and Alerting System, which is designed to monitor the environmental conditions, detect fire incidents in real time. Forest fires are among the primary environmental issues that can lead to serious consequences for wildlife, vegetation and human communities. Thus, early detection and quick response systems are crucial for limiting the damage (Dampage, et al., 2022).

This system was developed using an ESP32 microcontroller, MQ2 smoke sensor, flame sensor, GPS module, LED, and buzzer. Smoke sensor is used to detect smoke and toxic gasses; the flame sensor is used to detect infrared radiation given off by fire. The ESP32 decodes the data from the sensors and sends it via Wi-Fi via the MQTT communication protocol (Sharma, et al., 2023).

For real time data transmission and monitoring, it is implemented on the cloud with AWS EC2, Mosquitto MQTT broker, and Node-RED. If smoke/fire is detected, the system triggers local alerts and notifications via LED and buzzer and then sends an email alert through AWS Lambda and SNS services. The alert message also contains location data (latitude and longitude) to help expedite response and fire management (Rao, et al., 2022).

2.2. Design Diagrams

The design diagrams of the proposed system visually represent the system architecture, flow of data and connections between hardware components. These diagrams assist in visualising the interaction between the system components for real-time detection of fire and generation of fire alerts. The system design consists of an architecture diagram, block diagram, flowchart, circuit diagram and a schematic diagram, which represent different views of the system.

2.2.1. Architectural Diagram

The architecture of the smart forest fire detection and alert system is shown in the architectural diagram, following the layered model of the IoT. The layers include the

device, network, cloud and application layers. The device layer is made up of smoke sensor (MQ-2), flame sensor and GPS module (for location data). The sensors are connected to the ESP32 microcontroller which processes and transmits sensor data.

The network layer includes Wi-Fi and MQTT protocol, which allows the ESP32 to communicate with the cloud infrastructure. Cloud layer is done by running the Mosquitto MQTT broker on AWS EC2 instances and the Node-RED platform for real-time data monitoring and processing. The sensor data is transmitted to Node-RED via MQTT, and the alert workflow is managed by that.

The system activates local alerts when fire or smoke is detected (LED and buzzer). AWS Lambda and AWS SNS services are used in the application layer to send email notifications with fire alerts messages and location coordinates to the users and concerned authorities, for continuous monitoring and quick emergency response.

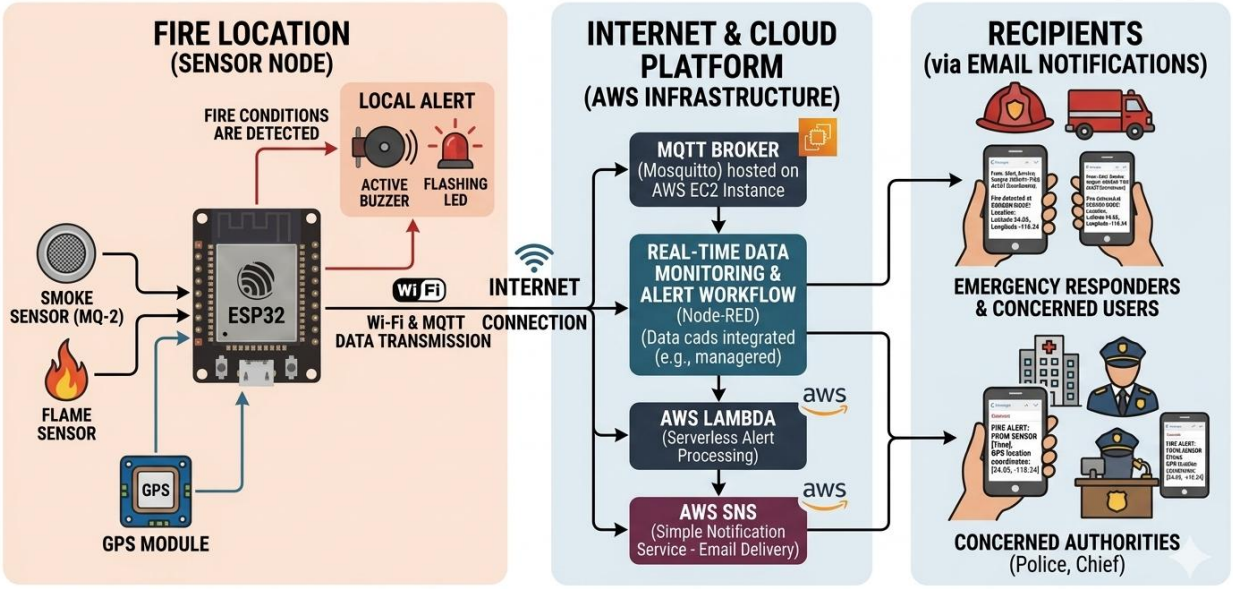

Figure 2: System Architectural Diagram

2.2.2. Block Diagram

The block diagram represents the overall workflow of the Smart Forest Fire Detection and Alert System. The system begins with the MQ-2 smoke sensor, flame sensor, and GPS module connected to the ESP32 microcontroller. The sensors continuously monitor environmental conditions and send sensor data to the ESP32 for processing.

The ESP32 activates local alert components such as LED and buzzer whenever smoke or fire is detected. Using Wi-Fi and MQTT protocol, the ESP32 transmits data to the Mosquitto MQTT broker hosted on AWS EC2 instance. Node-RED subscribes to the MQTT messages, processes the received data, and triggers the cloud alert mechanism.

AWS Lambda and SNS services are used to send real-time email notifications with fire alert information and location coordinates to users and authorities. The block diagram demonstrates the complete data flow from sensors to cloud services and emergency alert delivery.

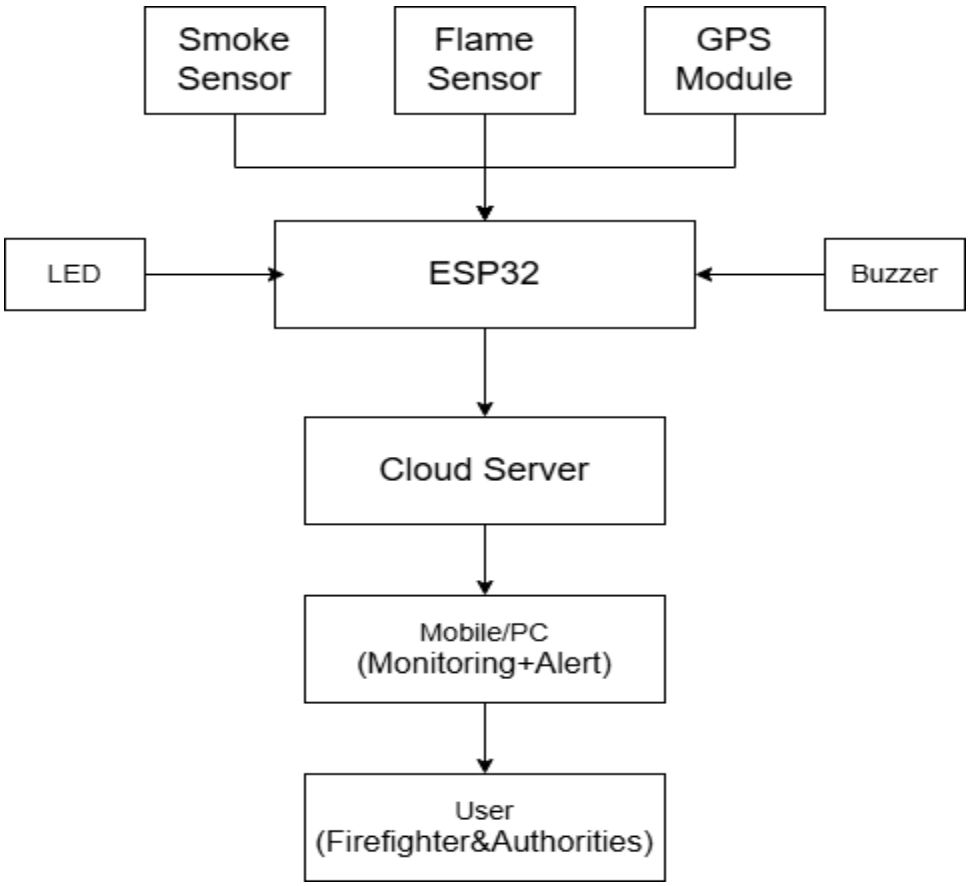

Figure 3: Block Diagram of the System

2.2.3. Flowchart

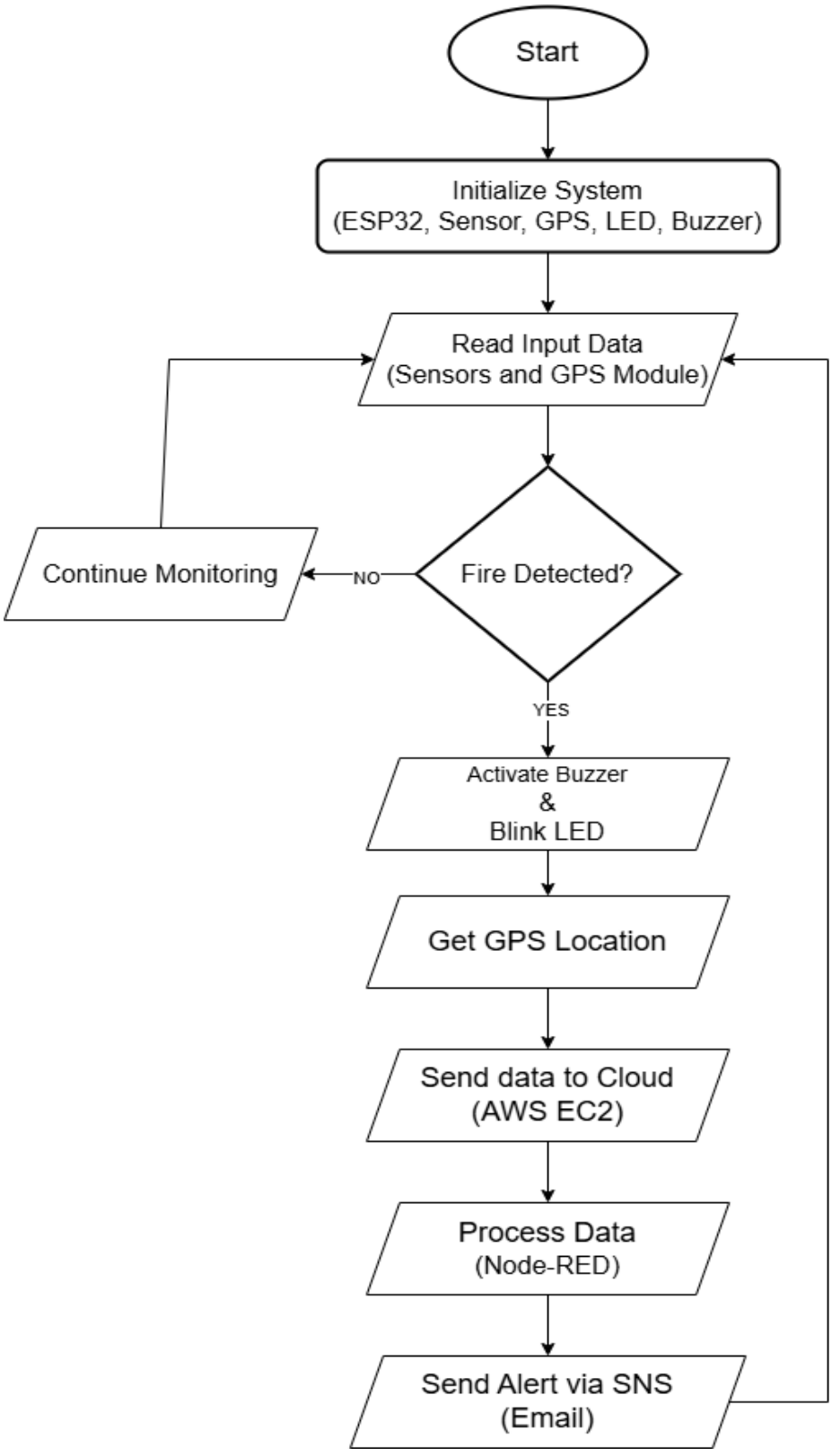

Figure 4: Flowchart

2.2.4. Circuit Diagram

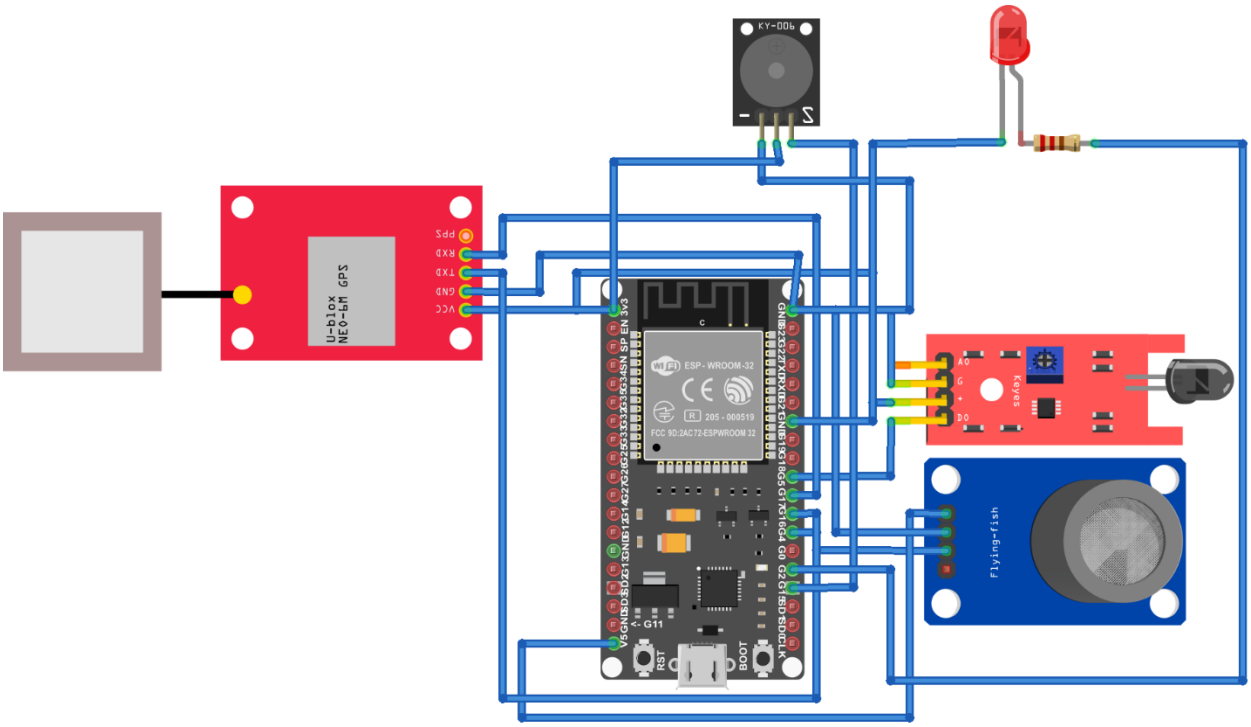

Figure 5: Circuit Diagram of Entire System

2.2.5. Schematic Diagram

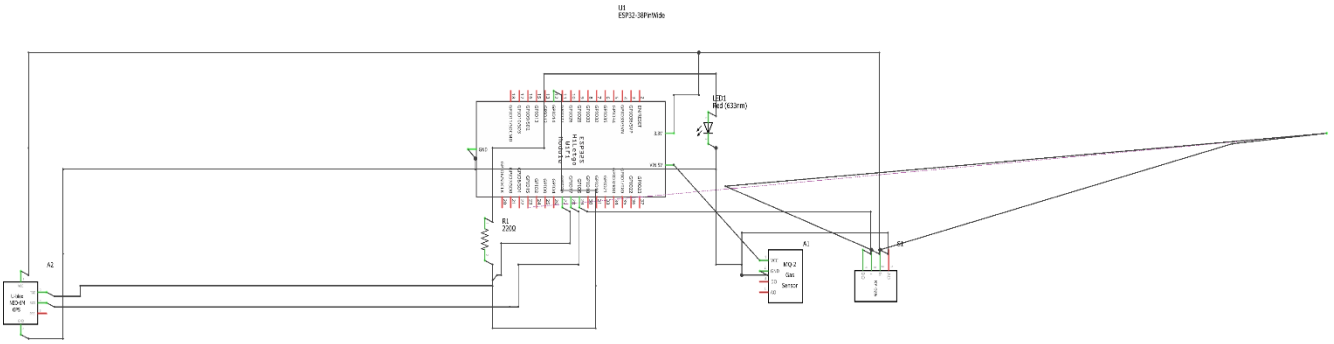

Figure 6: Schematic Diagram of the Entire System

2.3. Detailed Requirement Analysis

2.3.1. Hardware Components

The system has various hardware components that provide proper fire detection as well as reliable communication.

ESP32 Microcontroller: The ESP32 is the central processing unit of the system. It gathers data via sensors and then transmits them to the cloud via inbuilt Wi-Fi. It is selected because it is cheap, high-performing, and has inbuilt wireless connectivity (Babiuch, et al., 2019).

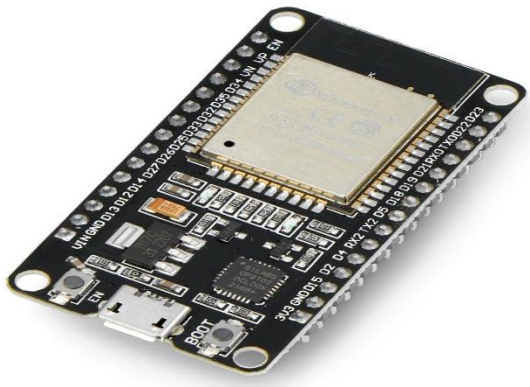

Figure 7: ESP32

MQ-2 Smoke Sensor: The smoke and flammable gases in the environment are detected by the MQ-2 sensor. The system is more reliable as it aids in the early detection of fire before it begins to spread.

Figure 8: MQ-2 Smoke Sensor

Flame Sensor: Flame sensor identifies fire radiation in the form of infrared. It is employed to verify the existence of flames and enhance accuracy of fire detection.

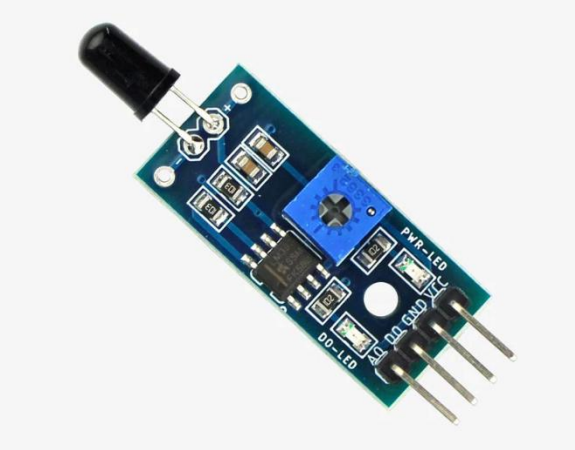

Figure 9: Flame Sensor

GPS Module (NEO-6M): The GPS module is used to provide location information in the form of latitude and longitude coordinates. During system implementation and testing, fixed GPS coordinates were temporarily used because of satellite connectivity limitations in the testing environment. The location information is included in alert notifications to help authorities identify the affected area quickly (JustDoElectronics, 2023).

Figure 10: GPS Module

Buzzer: The buzzer is used to give an audible signal that there is fire, and this will assist the people around the site to react promptly.

Figure 11: Buzzer

LED Indicator: The LED is a visual alert system, which shows fire detection.

Figure 12: LED

2.3.2. Software Components

The system is built with the help of different software tools so that the functionality is properly developed and integrated with the cloud.

Arduino IDE: The program code to the ESP32 microcontroller is written, compiled, and uploaded using Arduino IDE. It is also used to monitor and debug serial systems during system development.

Figure 13: Arduino IDE

MQTT Protocol: MQTT (Message Queuing Telemetry Transport) is used as the communication protocol between the ESP32 and cloud infrastructure. MQTT is lightweight and efficient and can be used for real-time IoT applications (Usmani, 2021).

Mosquitto MQTT Broker: The Mosquitto MQTT broker is running on an AWS EC2 instance and used to receive and process the MQTT messages that are sent by the ESP32 device.

Node-RED: Real-time data processing, workflow management and cloud-to-MQTT communication integration is handled by Node-RED. It listens to MQTT topics, analyses the sensor information and activates alert workflows (Uzougbo, et al., 2024).

AWS EC2: To host Mosquitto MQTT broker and Node-RED services for remote monitoring and communication use the Amazon EC2 service as the cloud computing platform (Sambasivarao, et al., 2024).

AWS Lambda: AWS Lambda is the cloud function used to process the alert requests received from Node-RED, to trigger the notification service, which is served in a serverless way.

AWS SNS (Simple Notification Service): During fire detection events, real-time ecosystem alerts are sent to users and relevant authorities via AWS SNS containing fire alert messages and location coordinates.

2.4. Individual Contribution Plan

2.4 Individual Contribution Plan

Ujwal Sharma played the major role in the overall development and implementation of the Smart Forest Fire Detection and Alert System. His contributions included ESP32 programming, MQTT communication setup, AWS EC2 configuration, Node-RED workflow development, AWS Lambda and SNS integration, system testing, troubleshooting.

Smith Bhattarai contributed significantly to the hardware integration and implementation process. His responsibilities included sensor interfacing, circuit setup, cloud communication testing, troubleshooting, and assisting in the practical development of the system, and major report documentation.

Bibek Dangi assisted in hardware assembly, complete system integration, flame and smoke sensor testing, LED and buzzer implementation, and practical testing of the final prototype during real-time demonstrations.

Abhinav Shrestha contributed by assisting in literature review, information collection, and documentation support during report preparation.

Abinash Kumar Sah assisted in report formatting, figure arrangement, diagram preparation, and reference management.

Shital Kapari supported the project through background research, proofreading, and minor documentation-related tasks.

Although individual members had different responsibilities during various stages of the project, all major decisions, implementation activities, testing procedures, and final report preparation were completed collaboratively as a team.

3. Development

3.1. Planning and Design

The project started with the selection of a problem which can be solved through IoT and cloud technologies. Forest fire detection was chosen because of the devastating effects of forest fires, and the inefficiency of traditional detection techniques.

A preliminary discussion was conducted to assess various IoT project topics based on feasibility, cost, availability of components and implementation. After this discussion, the project idea of developing a Smart Forest Fire Detection and Alert System was confirmed, as it has both practical applications and incorporates IoT and cloud computing technologies.

After finalising the project topic, a literature review was done on fire detection systems and IoT-based monitoring systems. This provided insights into how sensors, microcontrollers and the cloud can be combined to form an automatic detection system. This informed the design of a simple system structure.

The system includes environmental sensors (MQ-2 smoke sensor and flame sensor) for fire detection, ESP32 microcontroller for sensor data processing, GPS module for location information, and a cloud-based infrastructure using MQTT, AWS EC2, Node-RED, AWS Lambda and SNS for real-time monitoring and alert generation. The ESP32 was chosen for its Wi-Fi module, enabling direct connection to the cloud.

Prior to implementation, the system logic was designed and planned, which involved acquiring sensor data, detecting via threshold, generating alarm, and communicating with the cloud. This helped in ensuring that all parts will be integrated seamlessly as part of an IoT system.

In this way, the design and planning phase set the stage for the successful implementation of the Smart Forest Fire Detection and Alert System.

3.2. Resource Collection

Following the design and planning phase, the next phase was gathering the necessary hardware and software resources for the system implementation.

Following the architecture design, the main hardware components needed for this project include: ESP32 microcontroller, MQ-2 smoke sensor, flame sensor, LED indicator, buzzer and wires. These elements were chosen for their affordability, accessibility and applicability in IoT systems.

The ESP32 microcontroller was selected as the main controller for the system due to its in-built Wi-Fi module, which allows easy integration with the cloud platforms without the need for extra Wi-Fi modules. The MQ-2 smoke sensor was chosen to detect the presence of smoke and other gases in the air, enabling early fire detection. The flame sensor was employed to detect the infrared signal emitted by fire, thereby enhancing the system's reliability. The GPS module was included to provide location information during fire detection events. During implementation and testing, fixed GPS coordinates were temporarily used because of GPS satellite connectivity limitations in the testing environment.

The LED and the buzzer were used as output devices to notify the user when fire is detected. This allows local users to be immediately alerted, even in the absence of a cloud system.

As for the software tools, Arduino Integrated Development Environment (IDE) was chosen to develop, compile and upload the program into the ESP32 microcontroller. Amazon Web Services was chosen as the cloud service to facilitate real-time data sharing, remote monitoring and alerting. AWS IoT Core offers secure communication via MQTT and can be used to deploy scalable IoT systems.

MQTT protocol was selected for lightweight and efficient real-time communication between the ESP32 and the cloud infrastructure. A Mosquitto MQTT broker hosted on AWS EC2 instance was used to receive and manage sensor data transmitted from the ESP32. Node-RED was selected for workflow management, message processing, and integration between MQTT communication and cloud services. AWS Lambda and SNS

services were used to implement automated email alert notifications during fire detection events.

The components were gathered from various sources including personal hardware kits, college laboratories and electronic shops. The availability of the components ensured a timely system implementation.

This step ensured that all resources were ready for use and this enabled an efficient start for the system development and implementation phase.

3.3. System Development

Step 1: ESP32 Setup and Initial Power Verification

The ESP32 microcontroller was connected to a computer using a USB cable to provide power and enable program uploading. The onboard LED indicators were observed to confirm that the board was receiving power and functioning correctly. This step ensured that the ESP32 was properly initialized before integrating other components.

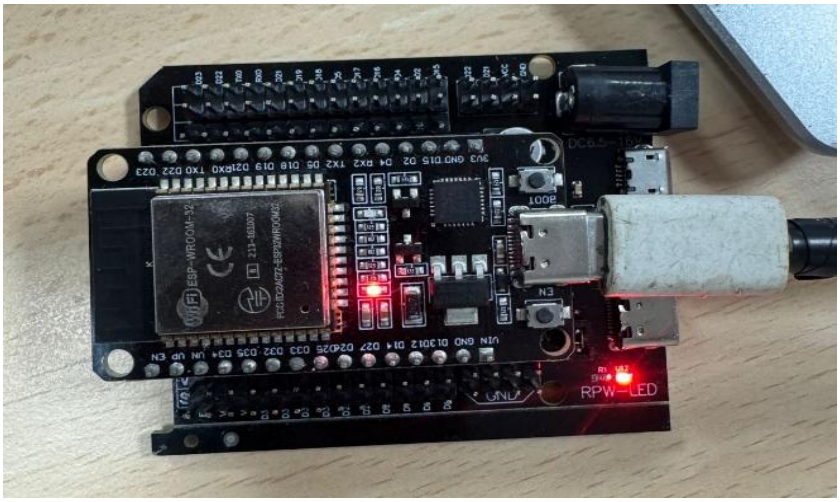

Figure 14: ESP32 connected to power source and initialized for development

Step 2: Breadboard Setup and Power Distribution

A breadboard was introduced to facilitate organized circuit connections. The ESP32 was connected to the breadboard using jumper wires to distribute power (VCC and GND) across the board. This setup provided a structured platform for connecting sensors and other components in subsequent stages.

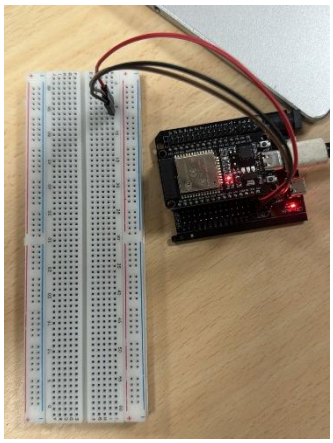

Figure 15: Breadboard setup for power distribution

Step 3: MQ-2 Smoke Sensor Integration

The MQ-2 smoke sensor was connected to the ESP32 to detect smoke and gas in the environment. The sensor requires a short warm-up period to provide stable readings. It was tested using real smoke to verify proper detection. **Connection:**

- VCC → 5V
- GND → GND
- DO → GPIO 4

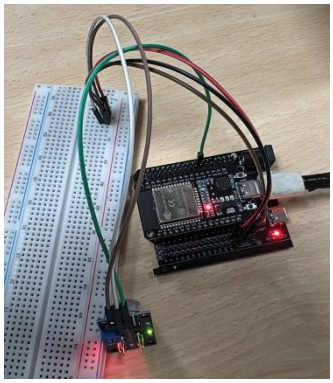

Figure 16: MQ-2 sensor connected to ESP32 for smoke detection

Step 4: Flame Sensor Integration

The flame sensor was connected to the ESP32 to detect fire using infrared light emitted by flames. The sensor was tested by bringing a flame close to verify its response. Sensitivity was adjusted using the onboard potentiometer to ensure accurate detection. **Connection:**

- VCC → 3.3V

- GND → GND
- DO → GPIO 5

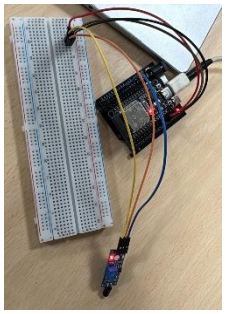

Figure 17: Flame sensor connected to ESP32 for fire detection

Step 5: GPS Module Integration

The GPS module (NEO-6M) was connected to the ESP32 to obtain real-time location data such as latitude and longitude. UART communication was used for data transmission between the GPS module and ESP32. The module was tested by placing it near an open area to acquire satellite signals and verify location output.

Connection:

- VCC → 3.3V
- GND → GND
- TX → GPIO 22
- RX → GPIO 23

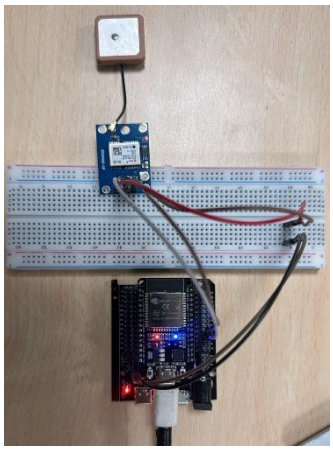

Figure 18: GPS module connected to ESP32 for real-time location tracking

Step 6: Buzzer Integration

A buzzer was connected to the ESP32 to provide an audible alert when fire or smoke is detected. The buzzer was controlled using a GPIO pin and programmed to activate when the system detects hazardous conditions. This ensured immediate local warning in addition to remote alerts.

Connection:

- VCC → 3.3V
- GND → GND
- I/O → GPIO 15

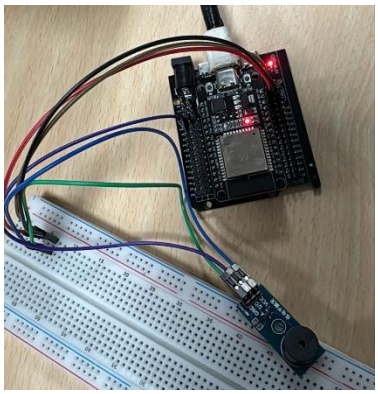

Figure 19: Buzzer connected to ESP32 for audible alert system

Step 7: LED Integration

An LED was connected to the ESP32 to provide a visual indication of fire detection. A 330Ω resistor was used to limit current and protect the LED. The LED was programmed to turn on or blink when fire or smoke is detected, providing a clear visual alert.

Connection:

- GPIO 2 → 330Ω Resistor → LED (+)
- LED (–) → GND

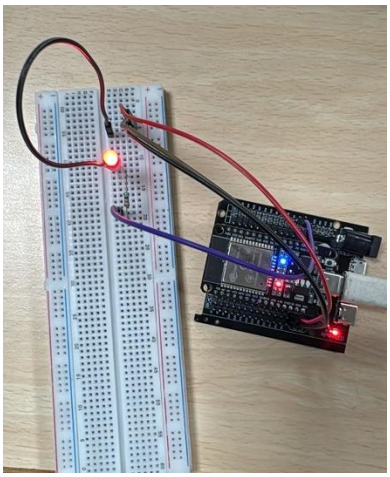

Figure 20: LED connected to ESP32 for visual alert indication

Step 8: Wi-Fi and MQTT Communication Setup

The ESP32 was programmed to join a Wi-Fi network, which would allow for cloud communication. Lightweight and efficient real-time data transmission is provided by the MQTT protocol. The ESP32 continuously checked the values of the smoke and flame sensor and sent alert messages to the MQTT topic if fire or smoke is detected.

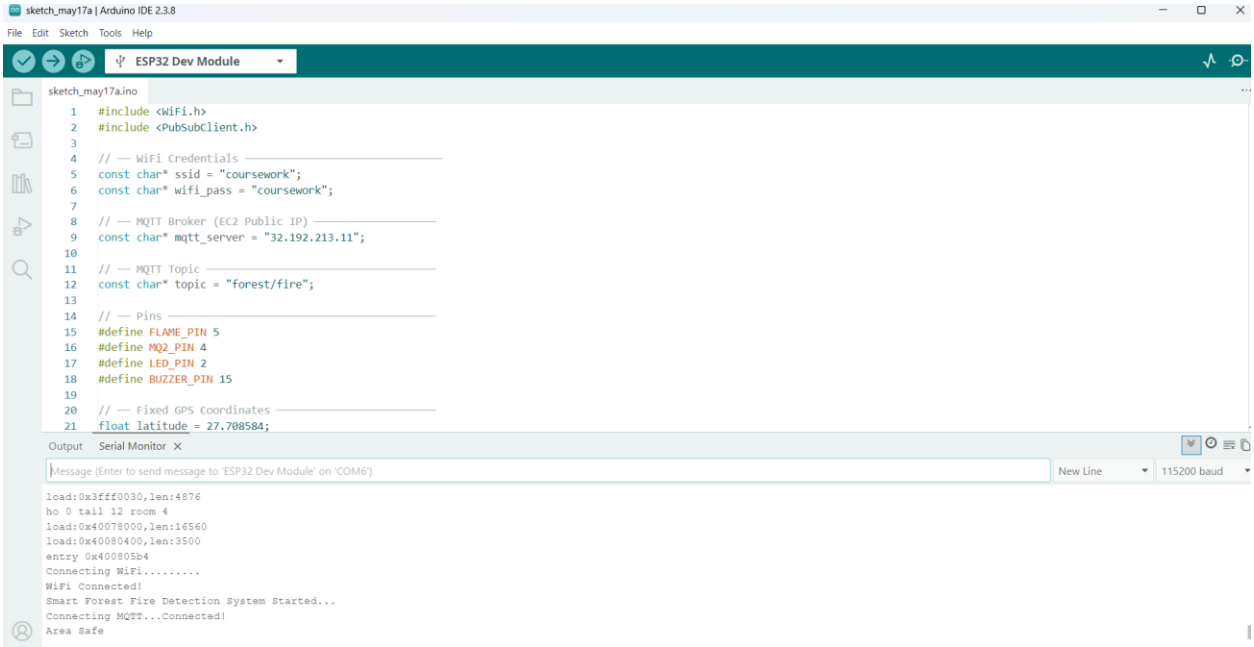

Figure 21: ESP32 Wi-Fi and MQTT Configuration for Real-Time Fire Alert Communication

Step 9: AWS EC2 and Mosquitto Broker Setup

An AWS instance (EC2) was launched to run the Mosquitto MQTT broker and the Node-RED platform. MQTT rules were created to open port 1883 and Node-RED rules to open port 1880. The broker MQTT was installed and configured to listen to the MQTT messages sent from the ESP32 device.

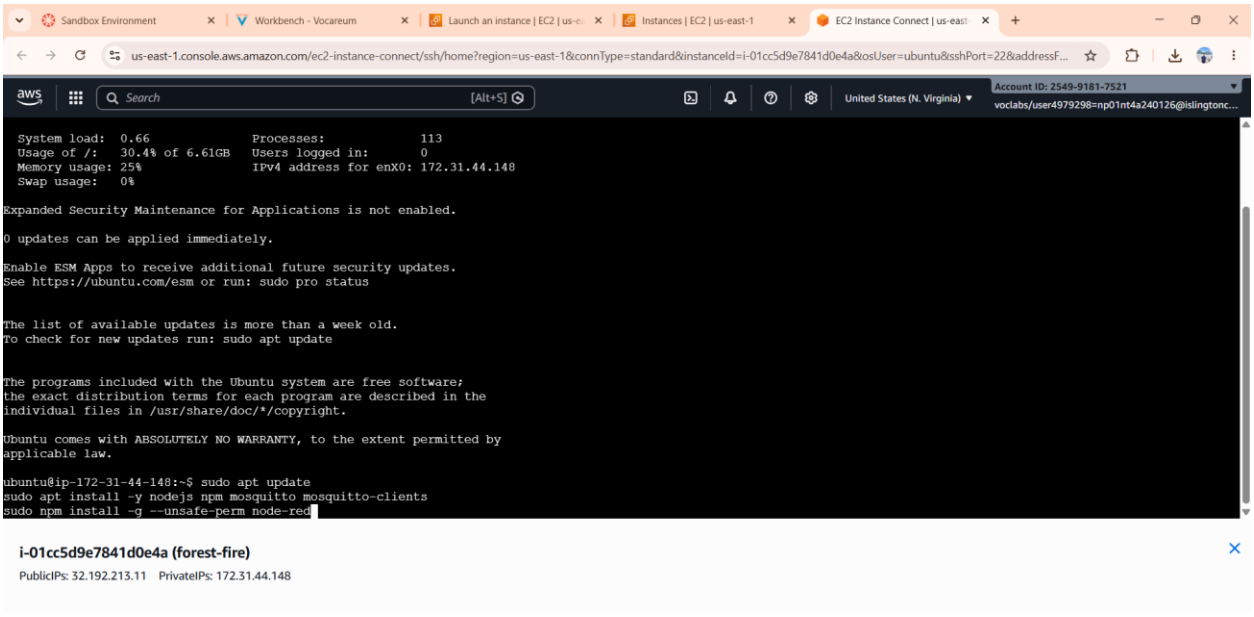

Figure 22: AWS EC2 Instance Hosting Mosquitto MQTT Broker and Node-RED Platform

Step 10: Node-RED Workflow Development

The ESP32 was connected to the AWS EC2, and Node-RED was installed and configured on the AWS EC2 instance to process the MQTT messages received from the ESP32. The fire alert topic was subscribed using MQTT input nodes. The fire alert topic was subscribed by using the MQTT input nodes. Processing alert conditions was done using function nodes and integration with AWS Lambda for cloud-based alert generation using the HTTP request nodes. For system testing and troubleshooting the debug nodes were also utilized.

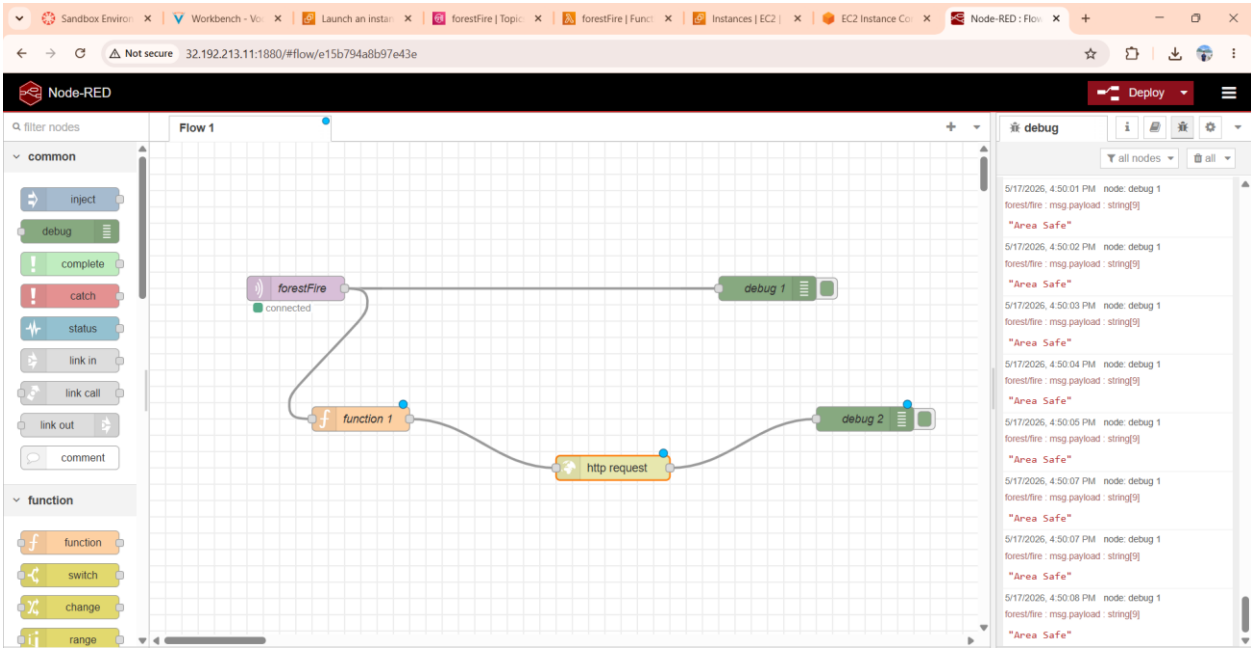

Figure 23: Node-RED Workflow for Processing MQTT Fire Alert Messages and Cloud Notification Integration

Step 11: AWS Lambda and SNS Integration

AWS Lambda was connected to handle the fire alerts received by Node-RED. The Python language was used to create a Lambda function to activate the AWS SNS notification service when fire was detected. The SNS email subscription was set up to deliver real-time alerts via email with fire detection data and location information to users and authorities.

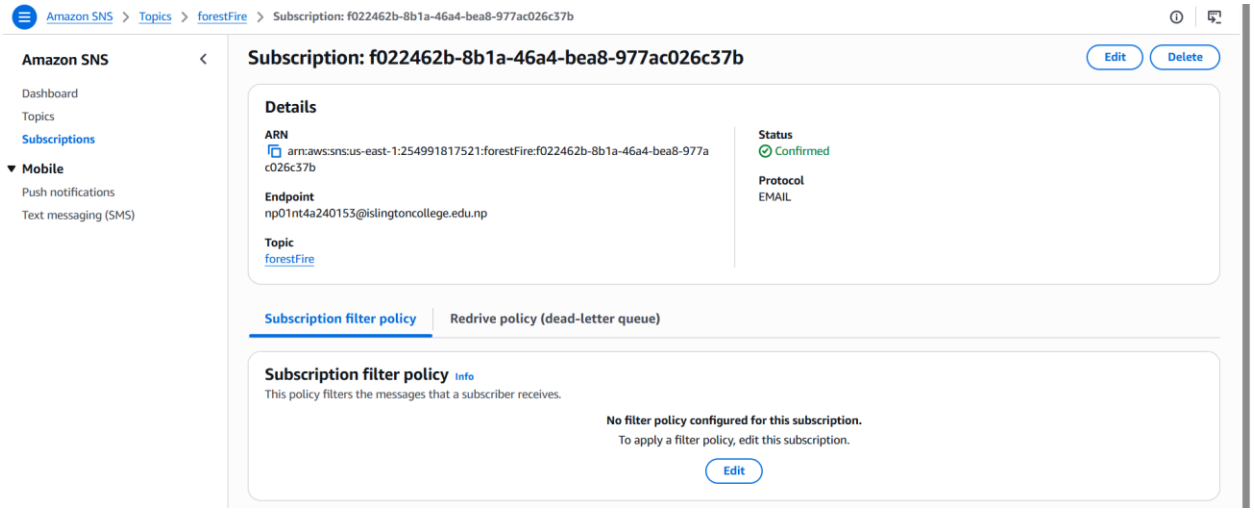

Figure 24: AWS SNS Email Subscription Configuration for Fire Alert Notification Service

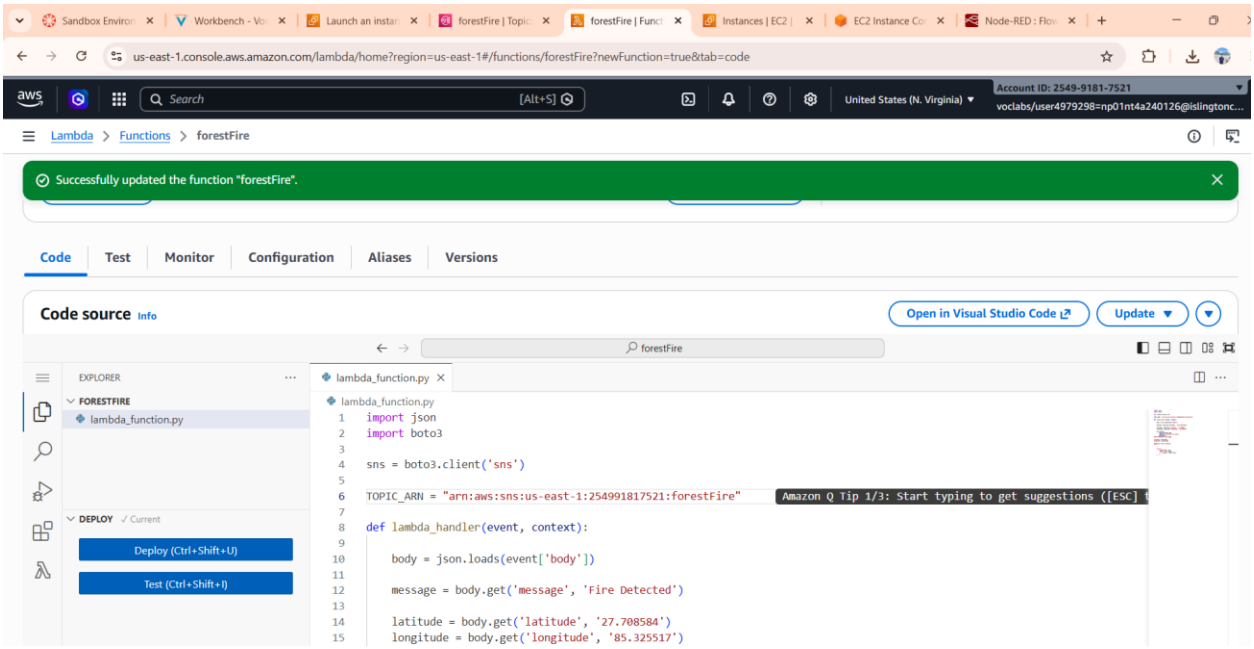

Figure 25: AWS Lambda Function Developed for Processing Fire Alert Notifications

Step 12: Final System Integration

The complete IoT-based Smart Forest Fire Detection and Alert System was successfully developed with all the hardware and cloud components. The ESP32 continuously monitored the environmental conditions by utilizing smoke and flame sensors and transmitted data of fire alerts by using MQTT protocol. LED and buzzer alerts were created locally, and cloud infrastructure with AWS EC2, Node-RED, Lambda and SNS allowed for remote monitoring and real-time email notifications when a fire was detected.

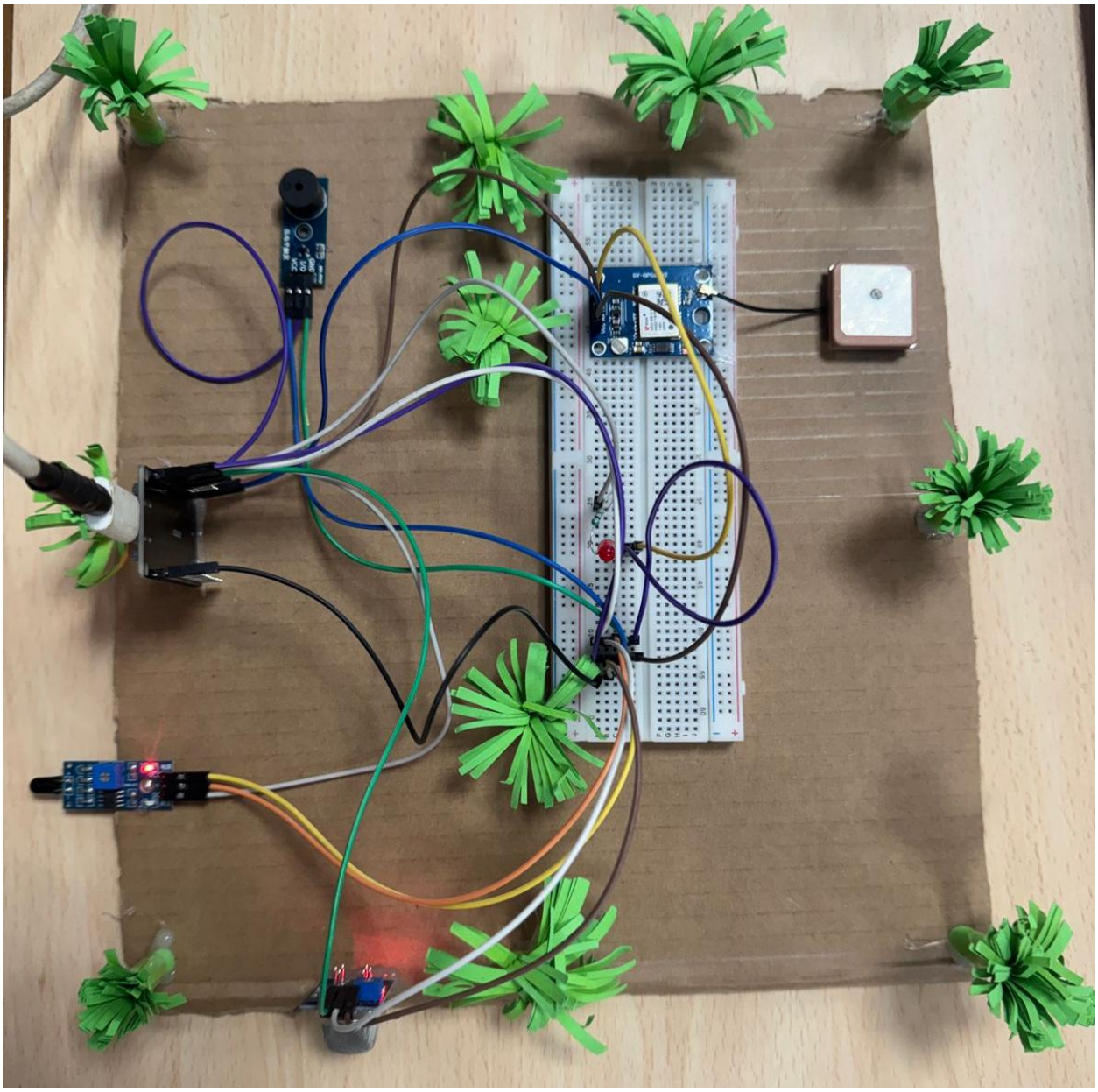

Figure 26: Complete Hardware and Cloud Integrated Smart Forest Fire Detection and Alert System Prototype

4. Result and Findings

4.1. Testing of Individual Components

4.1.1. Testing of Smoke Sensor

Test	1
Objective	To verify that the MQ-2 sensor can detect the presence of smoke accurately.
Action	The MQ-2 sensor was powered on and the code from Arduino IDE was compiled then smoke was introduced near the sensor using a lighter/incense stick.
Expected Output	The sensor should detect smoke and display a message such as “Smoke Detected” in the Serial Monitor.
Actual Output	The sensor successfully detected smoke and displayed the message “Smoke Detected” in the Serial Monitor.
Result	The test was successful. The Smoke Sensor worked.

Table 1: Working of the Smoke Sensor MQ-2

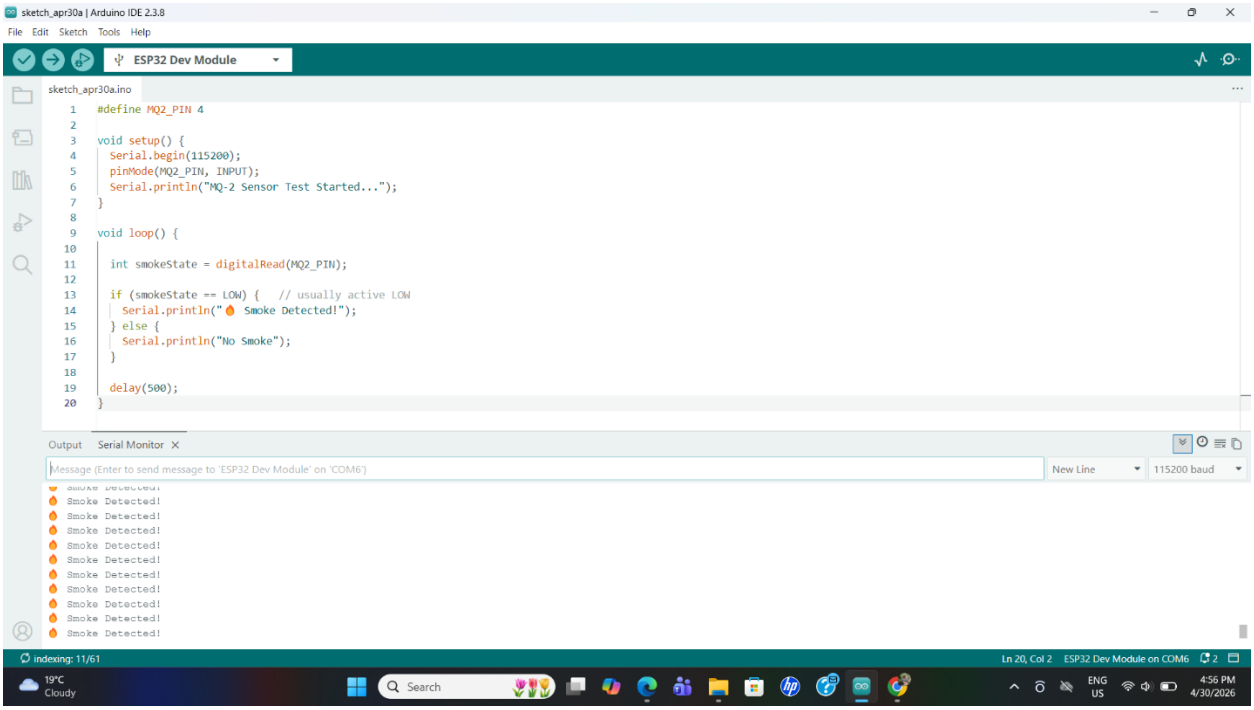

Figure 27: MQ-2 Smoke Sensor Code and Detection Output in Serial Monitor

Figure 28: MQ-2 Smoke Sensor Testing using Real Smoke

4.1.2. Testing of Flame Sensor

Test	2
Objective	To verify that the flame sensor can detect the presence of fire accurately.
Action	The flame sensor was powered on and connected to the ESP32. The code was uploaded using Arduino IDE, and a flame (lighter/matchstick) was brought close to the sensor.
Expected Output	The sensor should detect the flame and display a message such as “Flame Detected” in the Serial Monitor.
Actual Output	The sensor successfully detected the flame when brought close and displayed the message “Flame Detected” in the Serial Monitor.
Result	The test was successful. The Flame Sensor worked.

Table 2: Working of Flame Sensor

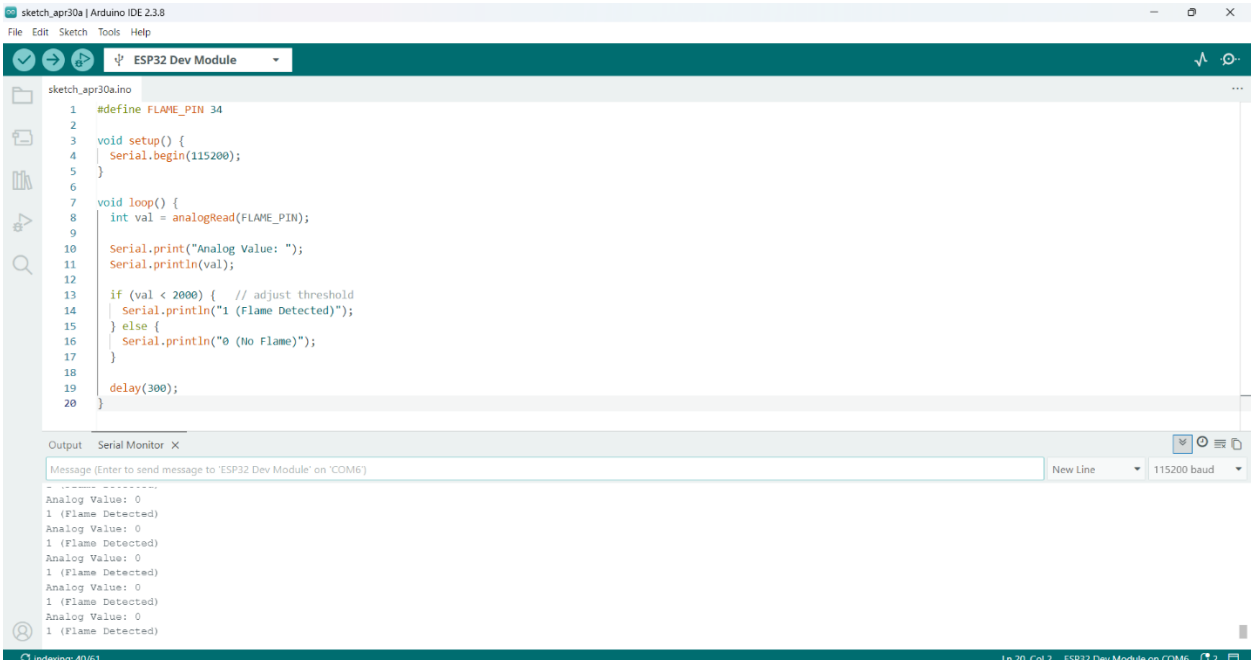

Figure 29: Flame Sensor Code and Detection Output in Serial Monitor

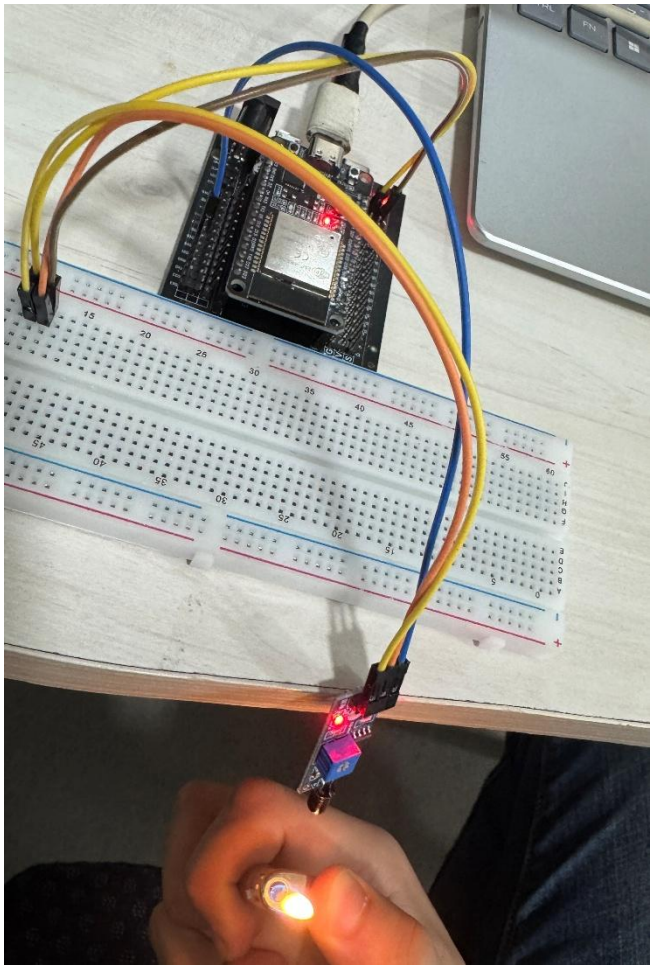

Figure 30: Testing of Flame Sensor using Fire Source

4.1.3. Testing of Buzzer

Test	3
Objective	To verify that the buzzer produces an audible alert when triggered by the ESP32.
Action	The buzzer was connected to the ESP32, and a test program was uploaded using Arduino IDE. The buzzer was activated through code to check if it produces sound when a signal is sent.
Expected Output	The buzzer should produce a continuous or intermittent sound when activated.
Actual Output	The buzzer successfully produced sound when triggered by the ESP32 through the program.
Result	The test was successful. The buzzer worked correctly and produced the required alert sound.

Table 3: Working of Buzzer

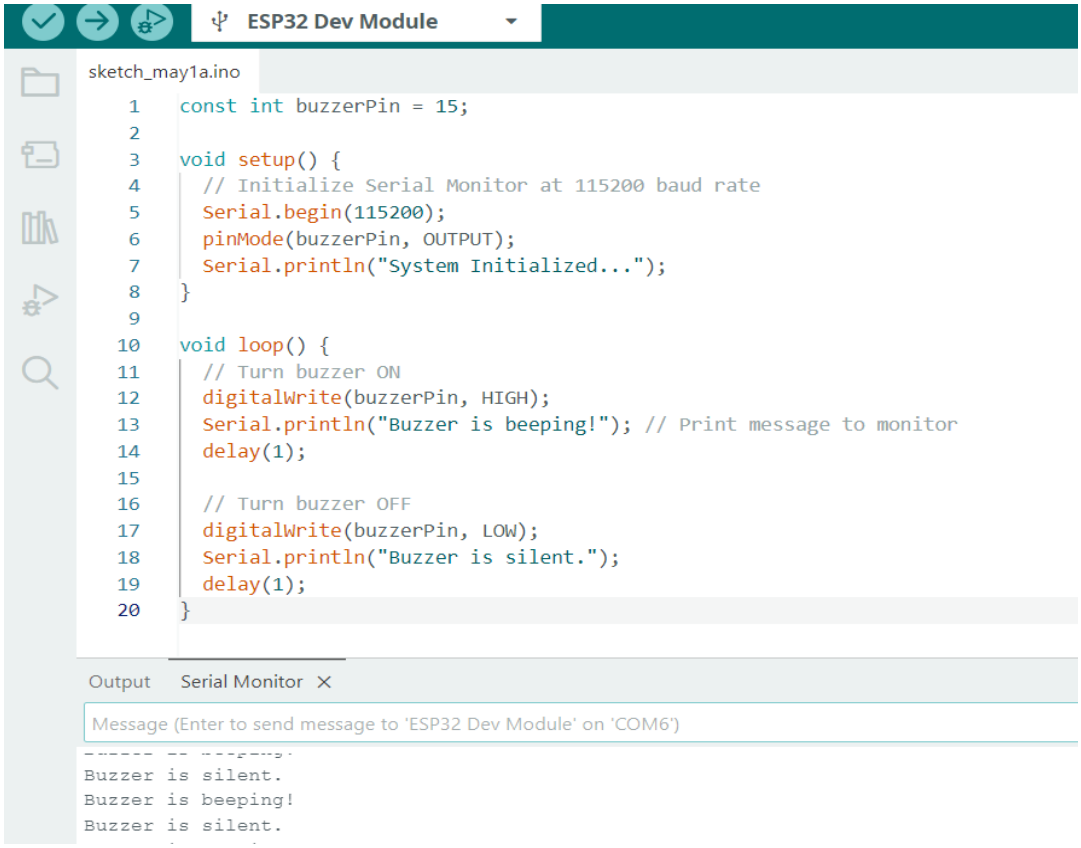

Figure 31: Buzzer Code and Detection Output in Serial Monitor

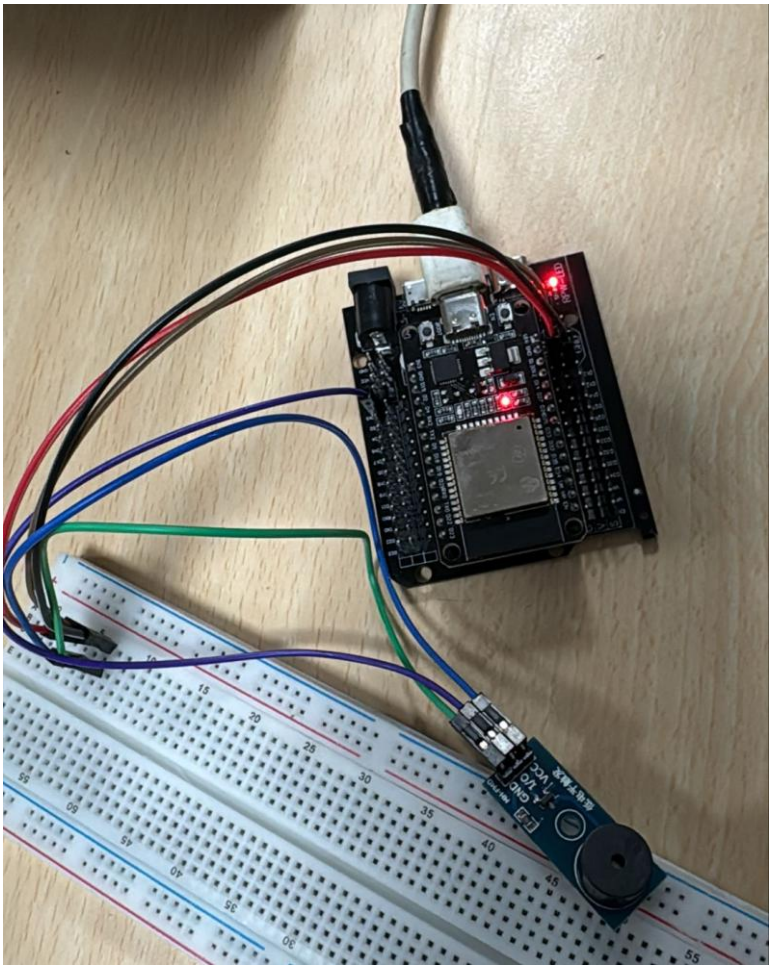

Figure 32: Buzzer Activation during Testing

4.1.4. Testing of LED

Test	4
Objective	To verify that the LED turns on and provides a visual indication when triggered by the ESP32.
Action	The LED was connected to the ESP32 using a 330Ω resistor. A test program was uploaded using Arduino IDE to turn the LED.
Expected Output	The LED should turn ON when a code sends the signal.
Actual Output	The LED successfully turned ON according to the program uploaded to the ESP32.
Result	The test was successful. The LED worked correctly and provided visual indication.

Table 4: Working of LED

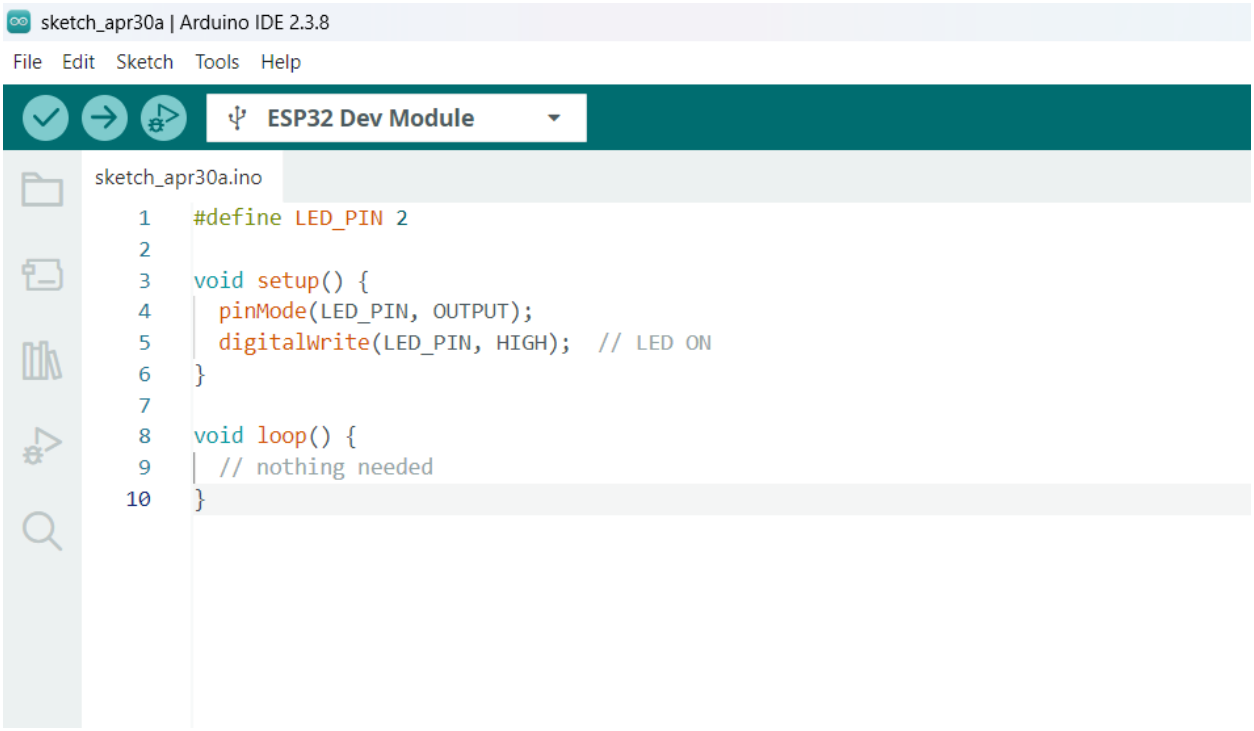

Figure 33: LED code for Blink

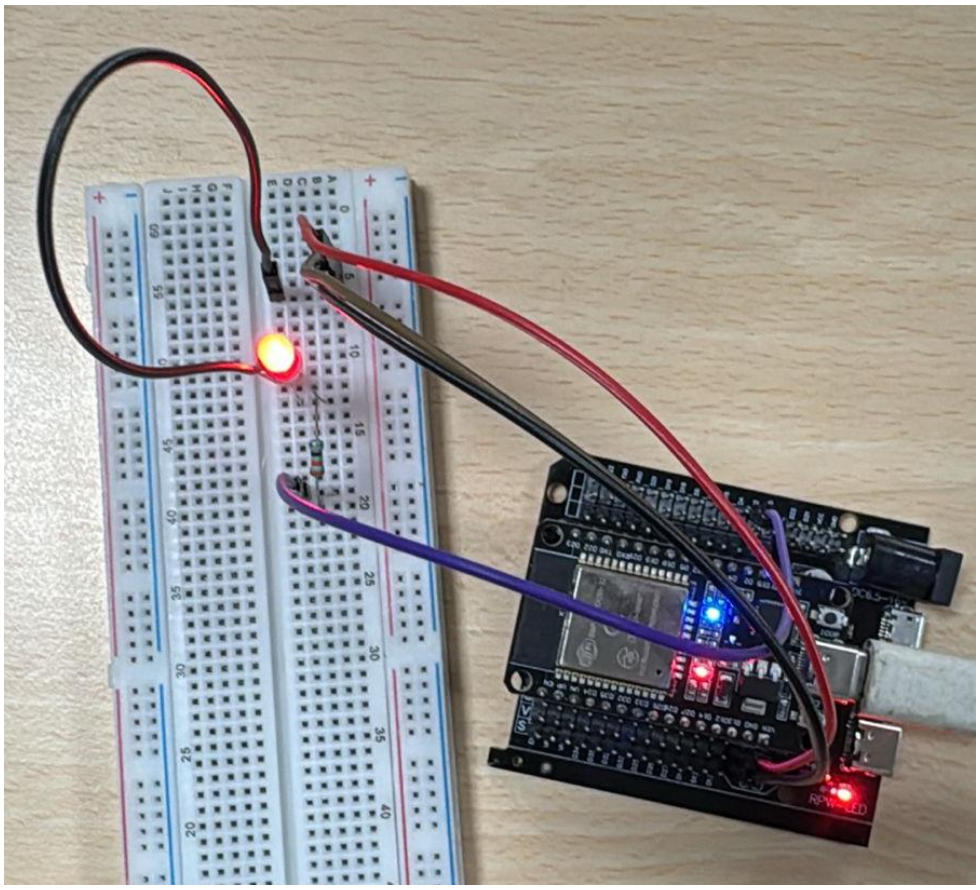

Figure 34: LED Activation during Testing

4.1.5. Testing of GPS Module

Test	5
Objective	To verify that the GPS module provides location coordinates and displays latitude and longitude values through the ESP32 serial monitor.
Action	The GPS module was connected to the ESP32 and programmed using Arduino IDE. Due to satellite connectivity limitations during indoor testing, fixed GPS coordinates were temporarily used for demonstration purposes. The ESP32 displayed the coordinates through the serial monitor.
Expected Output	The latitude and longitude coordinates should be displayed correctly in the serial monitor and included in the fire alert system.
Actual Output	The ESP32 successfully displayed the fixed latitude and longitude coordinates in the serial monitor during testing.
Result	The test was successful. The GPS location information was correctly displayed and integrated with the alert system.

Table 5: Testing of GPS Module

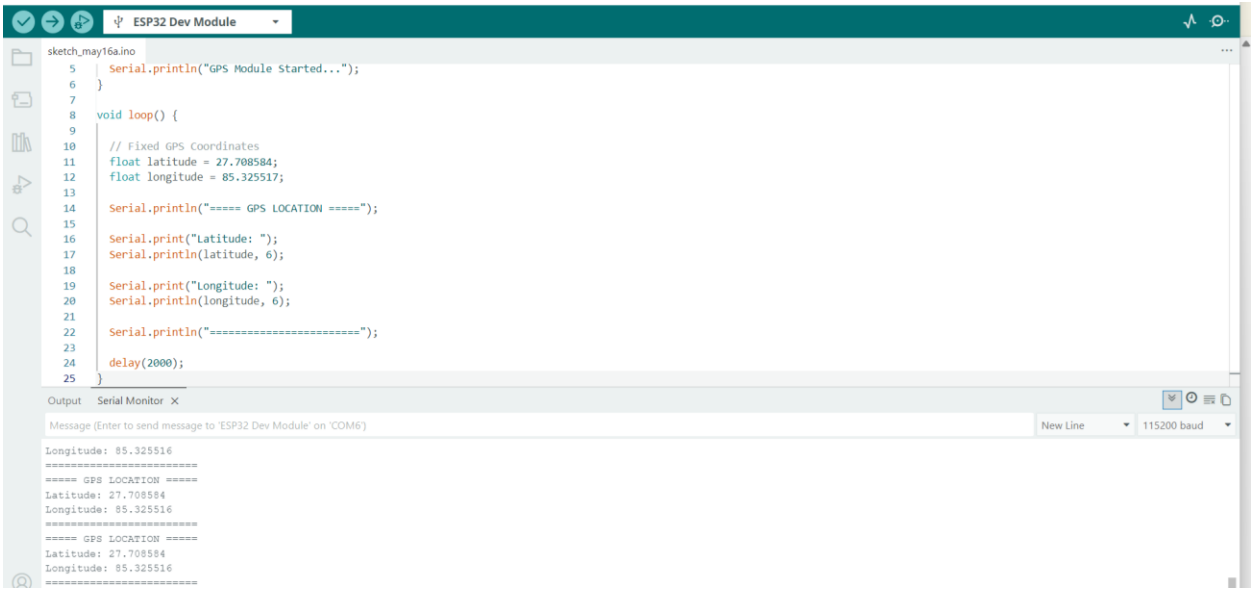

Figure 35: Testing of GPS Module and Display of Location Coordinates in Serial Monitor

4.2. Testing of Integrated System Locally

Test	6
Objective	To verify that the integrated system detects fire conditions and triggers local alerts using LED and buzzer.
Action	All components including the flame sensor, MQ-2 smoke sensor, LED, and buzzer were connected to the ESP32. A flame was introduced near the sensors using a lighter to simulate fire conditions.
Expected Output	When fire or smoke is detected, the LED should turn ON (or blink) and the buzzer should produce sound as an alert.
Actual Output	The sensors detected the presence of fire, and the ESP32 successfully activated the LED and buzzer as programmed.
Result	The test was successful. The system was able to detect fire conditions and generate local alerts effectively.

Table 6: Testing of Integrated System

Figure 36: Code for Integrated System

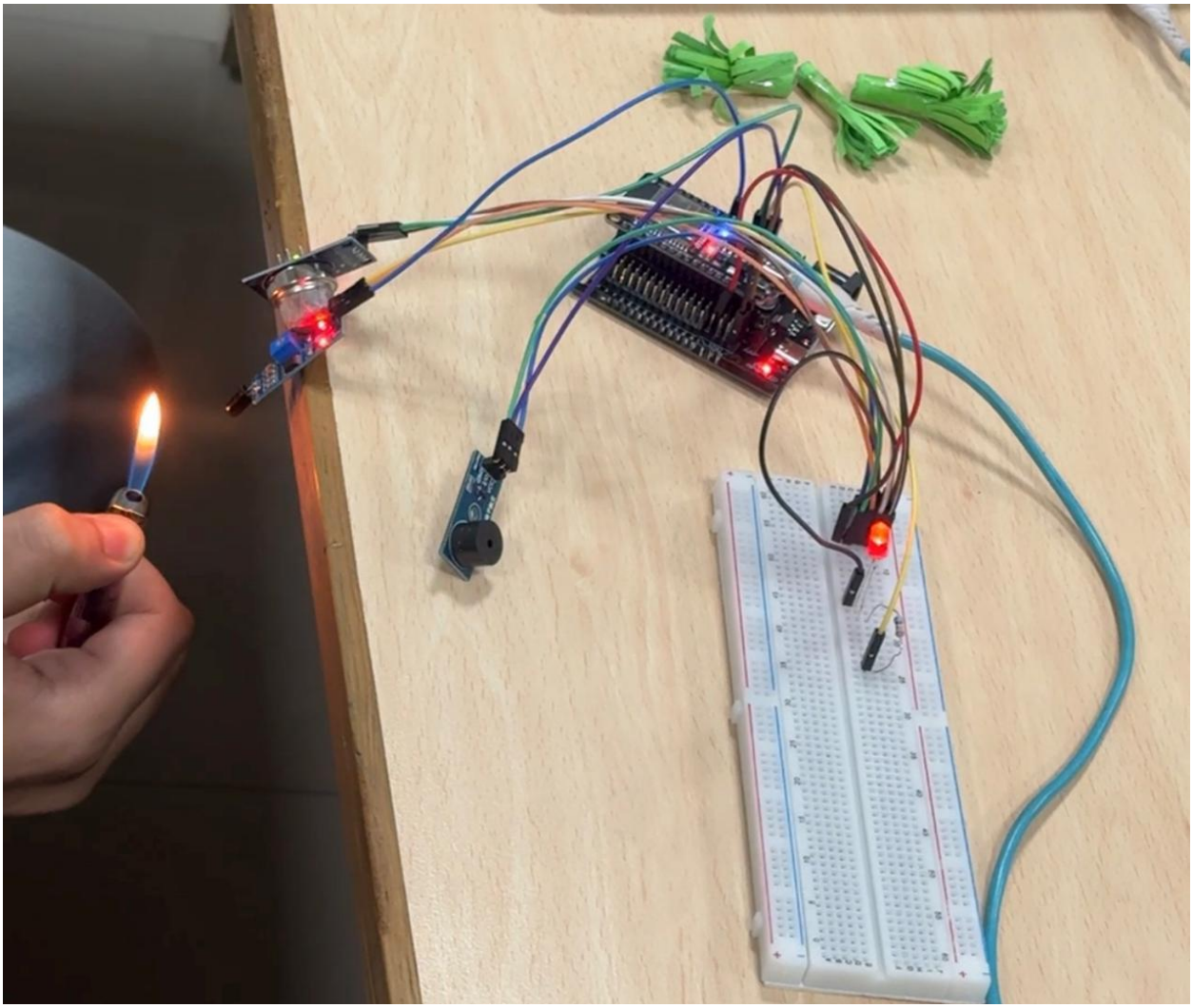

Figure 37: Testing of Buzzer and LED alert while fire detection

4.3. Testing of Integrated System with Email Alert

Test	7
Objective	To verify that the complete system can send email alerts through the cloud platform when fire conditions are detected.
Action	The ESP32 connected with the smoke sensor, flame sensor, GPS module, LED, buzzer, MQTT server, Node-RED, and AWS SNS service was tested under simulated fire conditions using smoke and flame near the sensors.
Expected Output	When fire or smoke is detected, the system should activate the LED and buzzer locally and send an email alert with fire notification and GPS location details to the registered user.
Actual Output	The system successfully detected fire conditions, activated the LED and buzzer, transmitted sensor data to the cloud server, and sent an email alert with location information to the registered email address.
Result	The test was successful. The overall integrated system worked correctly and generated real-time email alerts during fire detection.

Table 7: Testing of Integrated Smart Forest Fire Detection and Email Alert System

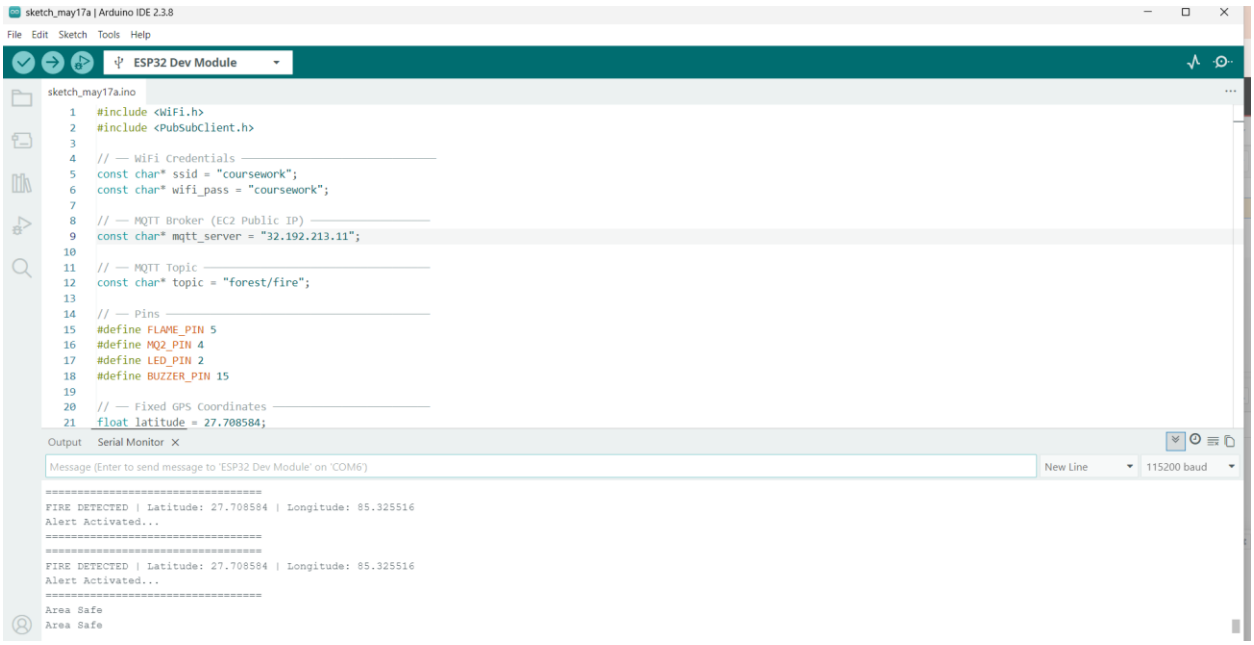

Figure 38: Integrated System Code and fire detected message in serial monitor

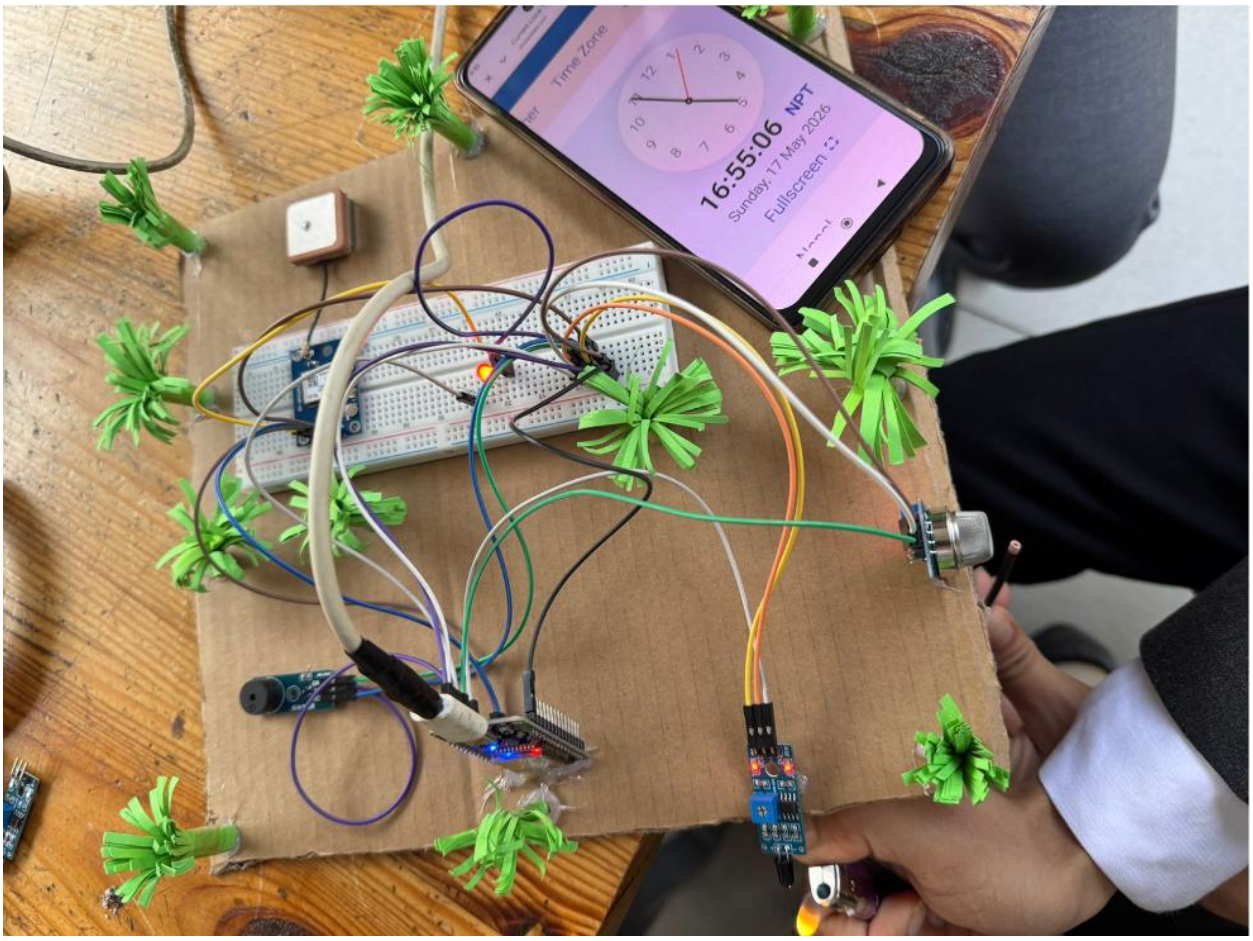

Figure 39: Final Prototype of Smart Forest Fire Detection and Alert System during Real-Time Fire Testing

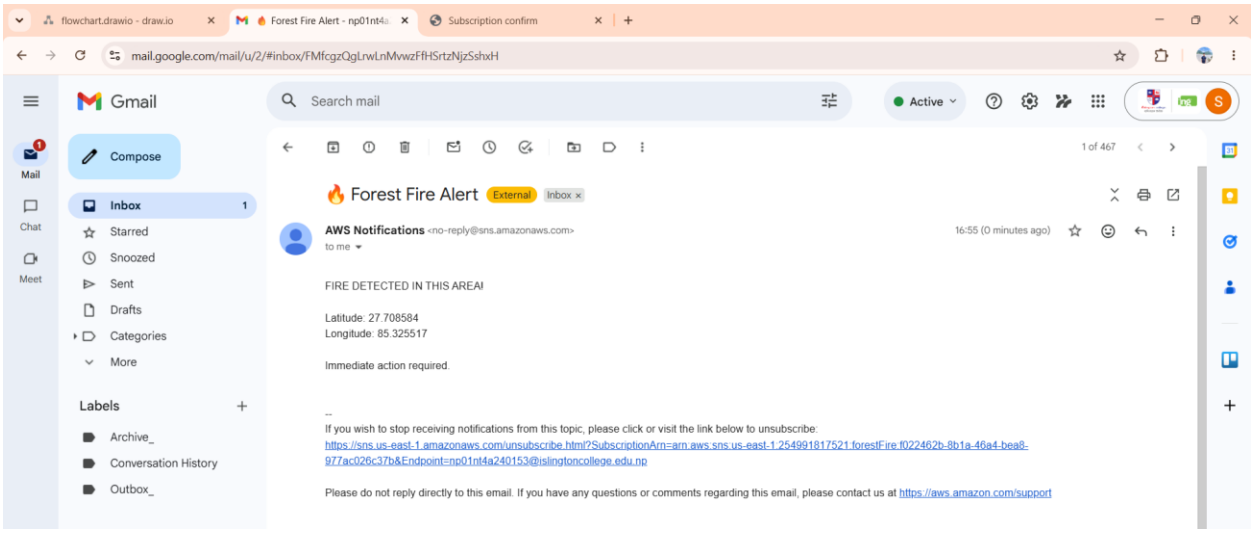

Figure 40: Email Alert Notification Generated through AWS SNS during Fire Detection Event

5. Future Work

The existing system shows a workable prototype that can be used to detect fire conditions and produce alerts based on sensors and embedded systems. Although the implementation achieves the main goals of fire detection and response, it is more of a prototype and cannot be further scaled or be as accurate and applicable to the real world.

Several improvements and extensions can be suggested to make the system more productive and fit better practical applications. Future improvements have been directed at the detection accuracy, broader system capabilities and the ability to deploy the system on a large scale in the real world. The list below outlines how further development of the system can be developed.

5.1 Advanced Sensor Integration

The system could be enhanced with more sophisticated and sensitive sensors in detecting fire and smoke. This would assist in decreasing false alarms and enhancing detection. Reliability can be further enhanced by combining several sensors and employing sensor fusion methods. Also, the use of calibration methods can be used to ensure the consistency in the performance with time.

5.2 Faster Response and Real-Time Processing

The system can also be optimized to improve the response time through optimization of the code and enhancement of speed of communication between components. More rapid data processing would enable a faster identification and alarms creation. It is possible to use real time processing methods in order to provide immediate response when there is an emergency. This will be important in reducing the impact of fire incidents.

5.3 Mobile Application Development

A special mobile application may be created to offer convenient interface to monitor the system. The application has the capability of showing real-time sensor data, fire alarms and location details. It may also have additional features like notification history and system status monitoring. This would render the system more user-friendly and convenient.

5.4 Large-Scale Deployment Capability

The system can be scaled up by installing several sensor nodes in a large forest. These nodes are able to communicate with one another and transmit data to a central server where it can be monitored. This would allow massive fire detection and enhance coverage. This network-based method would work better in a real forest scenario.

5.5 Environmental Data Integration

Other environmental variables like temperature, humidity, and wind speed can be incorporated in the system. These parameters are significant in the prediction and spread analysis of fires. The system can also give early warning and improved decision-making by analysing this data. This would improve the effectiveness of the fire detection system.

6. Conclusion

The Smart Forest Fire Detection and Alert System was successfully designed and implemented using IoT and cloud computing technologies. The system incorporates the main elements of the ESP32 microcontroller, flame sensor, MQ-2 smoke sensor, GPS module, LED, and the buzzer to identify the presence of fire and provide alerts. The process of development was structured hardware configuration, program coding and real-time testing, which showed that the theoretical content studied throughout the course was applicable in practice.

The system had the capability of detecting fire and smoke efficiently and give instant local notifications through visual and audible cues. Location tracking is made possible by the integration of the GPS module and makes the system more useful in real-life applications. There have been a few challenges in the process of implementation e.g. sensor calibration and acquisition of GPS signals but these challenges have been solved by means of testing and making modifications and now have a working prototype. The integration of MQTT communication, AWS EC2, Node-RED, AWS Lambda, and SNS services further enhanced the system by enabling cloud-based monitoring and real-time email alert notifications during fire detection events. The integration of MQTT communication, AWS EC2, Node-RED, AWS Lambda, and AWS SNS services enhanced the system by enabling cloud-based monitoring and real-time email alert notifications.

Altogether, the project demonstrates the opportunities of the IoT-based solutions in enhancing safety and the prompt detection of fire. As this system continues to be refined through the addition of new features like better sensor accuracy, real-time cloud connectivity, and large-scale implementation, it is possible to evolve the system into a dependable forest fire monitoring and disaster management tool. The hands-on experience has also helped me gain valuable insights in system design, integration and problem solving in this project.

7. References

Al-Fuqaha, A. et al., 2015. Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials,* 17(4), pp. 2347-2376.

Alkhatib, A. A., 2014. A Review on Forest Fire Detection Techniques. *International Journal of Distributed Sensor Networks,* 10(3), pp. 1-12.

Babiuch, M., Foltýnek, P. & Smutný, P., 2019. Using the ESP32 Microcontroller for Data Processing. *2019 20th International Carpathian Control Conference (ICCC),* 20(N.A.), pp. 1-6.

Dampage, U. et al., 2022. *Forest fire detection system using wireless sensor networks and machine learning,* Srilanka: Scientific Reports.

Jayavardhana, G., Buyya, R., Marusic, S. & Palaniswami, M., 2013. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems,* 29(7), pp. 1645-1660.

JustDoElectronics, 2023. *GPS NEO-6m Module.* [Online]
Available at: https://justdoelectronics.com/gps-neo-6m-module/
[Accessed 10 05 2026].

OSOYOO, 2021. *What is Blynk and How does it work?.* [Online]
Available at: https://osoyoo.com/2021/10/24/what-is-blynk-and-how-does-it-work/
[Accessed 10 04 2026].

Pandey, H. P. et al., 2022. Status and Practical Implications of Forest Fire Management in Nepal. *Journal of Forest and Livelihood,* 21(1), pp. 32-45.

Rao, N. P. et al., 2022. Early Warning Alarm System for Detecting Forest Fire Using NODE MCU with IoT. *International Journal of Innovative Research in Computer Science & Technology (IJIRCST),* 10(1), pp. 143-145.

Sambasivarao, L. V. et al., 2024. AWS And Future of Cloud Computing. *International Journal of Advanced Research in Computer and Communication Engineering,* 13(4), pp. 87-95.

Sharma, A. et al., 2023. An IoT-based forest fire detection system: design and testing. *Multimedia Tools and Applications,* 1(1), pp. 1-26.

Usmani, M. F., 2021. *MQTT Protocol for the IoT,* Hessen, Germany: Frankfurt University of Applied Sciences.

Uzougbo, O. I., Olanrewaju, . O. A. & Kayode, A. K., 2024. Node-RED and IoT Analytics: A Real-Time Data Processing and Visualization Platform. *A Subsidiary of Tech-Sphere Multidisciplinary International Journal (TSMIJ),* 1(1), pp. 1-12.